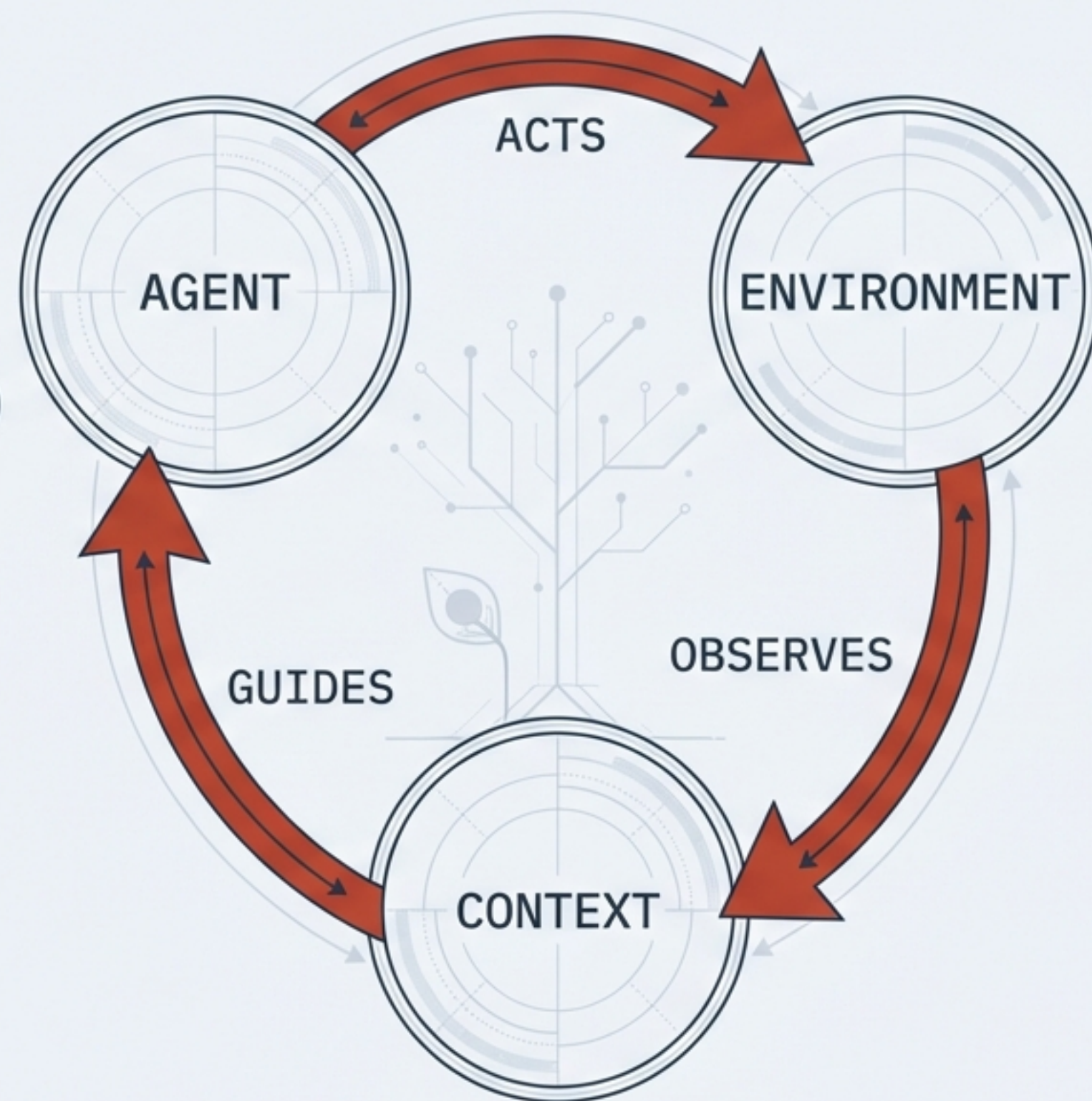


Agentic Context Engineering (ACE): วิวัฒนาการของบริบทเพื่อ โมเดลภาษาที่พัฒนาเองได้

เปลี่ยนจาก ‘การสรุปย่อ’
สู่ ‘คู่มือกลยุทธ์ (Playbook)’
ที่สมบูรณ์แบบ



Based on research from Stanford University & SambaNova Systems

การปรับแต่งโมเดลในปัจจุบันไม่ได้อยู่ที่การแก้ไขน้ำหนัก (Weights) แต่อยู่ที่การจัดการข้อมูลนำเข้า (Inputs)



ความเร็ว

สามารถเพิ่มความรู้ใหม่ได้ทันทีโดยไม่ต้องเทรนโมเดลใหม่ (Zero training time updates).



ความโปร่งใส

มนุษย์สามารถอ่านและตรวจสอบบริบทที่ถูกส่งเข้าไปในโมเดลได้ (Explainable & Interpretable).



ความสามารถในการแชร์

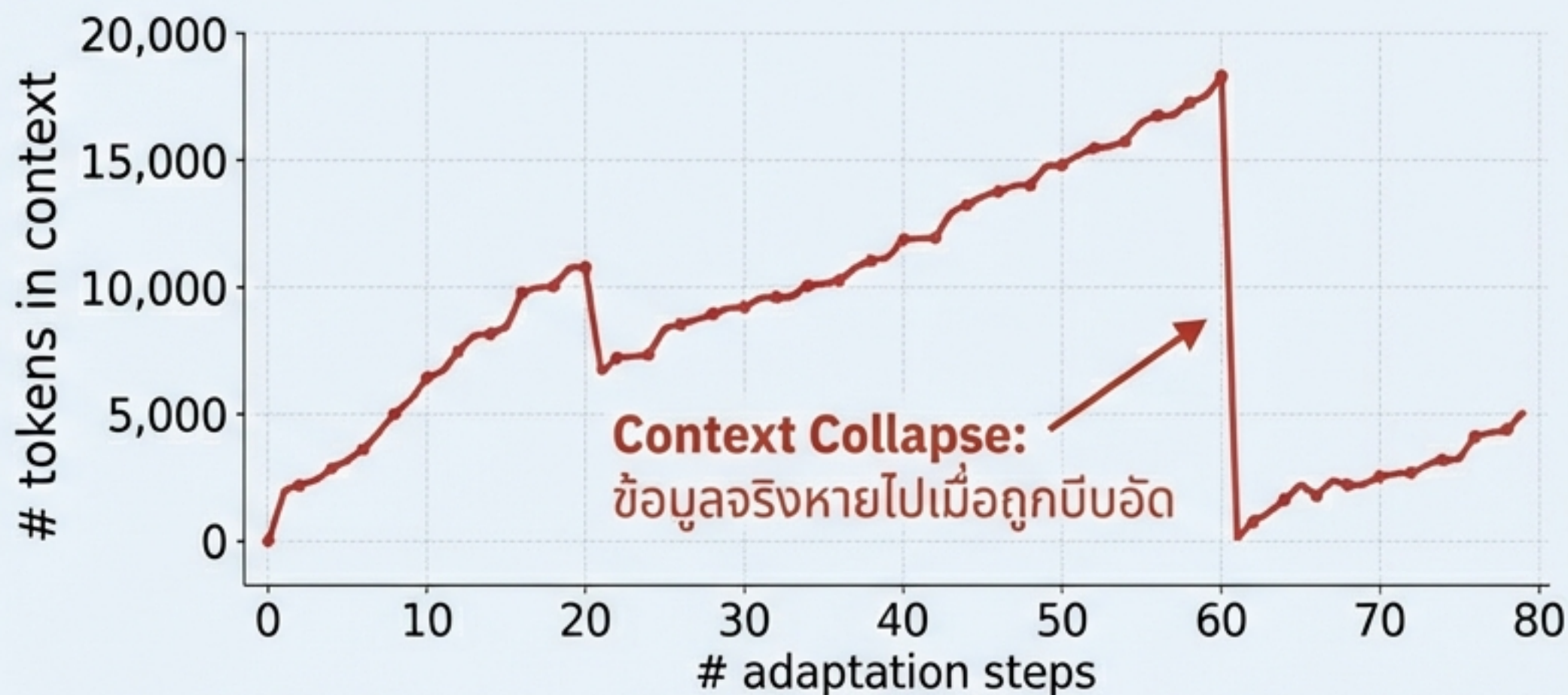
สามารถถ่ายโอนความรู้ระหว่างโมเดลหรือโมดูลต่างๆ ในระบบ Compound AI ได้

แต่เมื่อเราพยายามอัดเยียดข้อมูลจำนวนมากลงในบริบท เรากลับพบปัญหาใหญ่ที่ไม่คาดคิด...

อุปสรรคสำคัญ: 'Brevity Bias' และ 'Context Collapse'

Brevity Bias (กับटकความกระชับ)

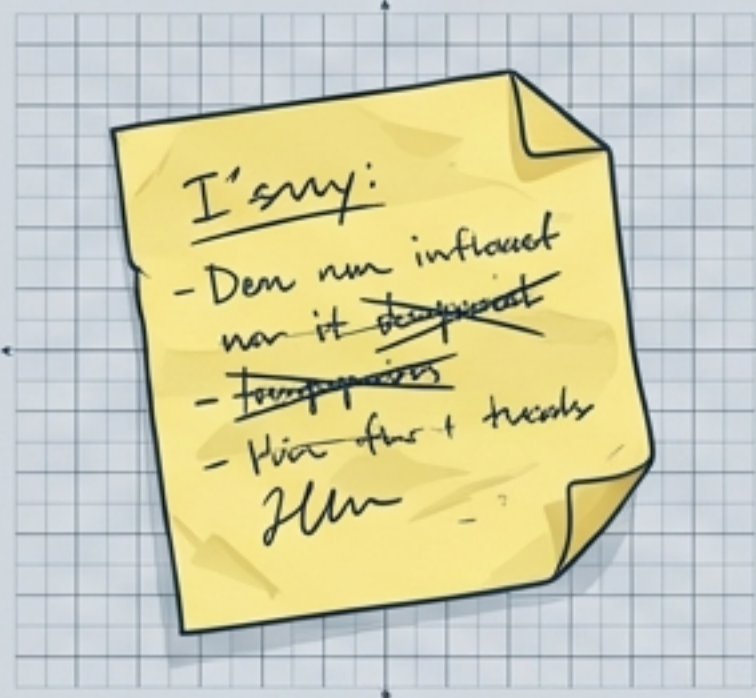
- เครื่องมือปรับแต่ง Prompt ส่วนใหญ่มักพยายาม 'ย่อความ' ให้สั้นที่สุด
- ผลลัพธ์: รายละเอียดสำคัญทางเทคนิค (Domain Nuance) และกฎเกณฑ์เฉพาะทางถูกตัดทิ้งไป



เมื่อให้ AI เขียนบริบทใหม่ซ้ำๆ ข้อมูลจะถูกบีบอัดจนสูญเสียความหมาย
(ความแม่นยำลดลงจาก 66.7% เหลือ 57.1%)

การเปลี่ยนผ่านกระบวนการทัศน์: เลิก “ย่อความ” แล้วหันมา “สะสมกลยุทธ์”

Old Way: Compression



พยายามสรุปทุกอย่างให้สั้นที่สุด
→ สูญเสียรายละเอียด
ข้อมูลไม่ครบถ้วน

New Way (ACE): Playbook



สะสม ปรับปรุง และจัดหมวดหมู่
(Accumulate, Refine, Organize)
→ รักษาความรู้เชิงลึกและตัวอย่างโค้ด

LLM ทำงานได้ดีกว่าเมื่อมีบริบทที่ละเอียดและครอบคลุม ไม่ใช่บทสรุปที่ถูกตัดตอน

ขอแนะนำ ACE: กรอบการทำงานสำหรับบริบทที่วิวัฒนาการได้

ACE (Agentic Context Engineering) คือเฟรมเวิร์กที่เปลี่ยนบริบทให้เป็น 'Playbook' ที่พัฒนาตนเองได้ ผ่านกระบวนการทำงานแบบ Agentic Loop



Offline Optimization

ปรับแต่ง System Prompt
ตั้งแต่ก่อนเริ่มใช้งาน



Online Memory

จดจำและเรียนรู้จาก
ประสบการณ์หน้างานจริง
(Test-time adaptation)

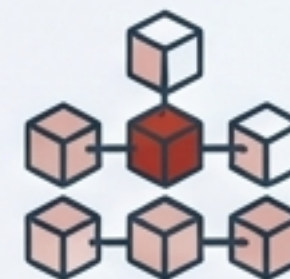


Self-Correction

เรียนรู้จากผลลัพธ์การทำงาน
(Execution Feedback)
โดยไม่ต้องการฉลาก
(Labels)



Context
Playbook

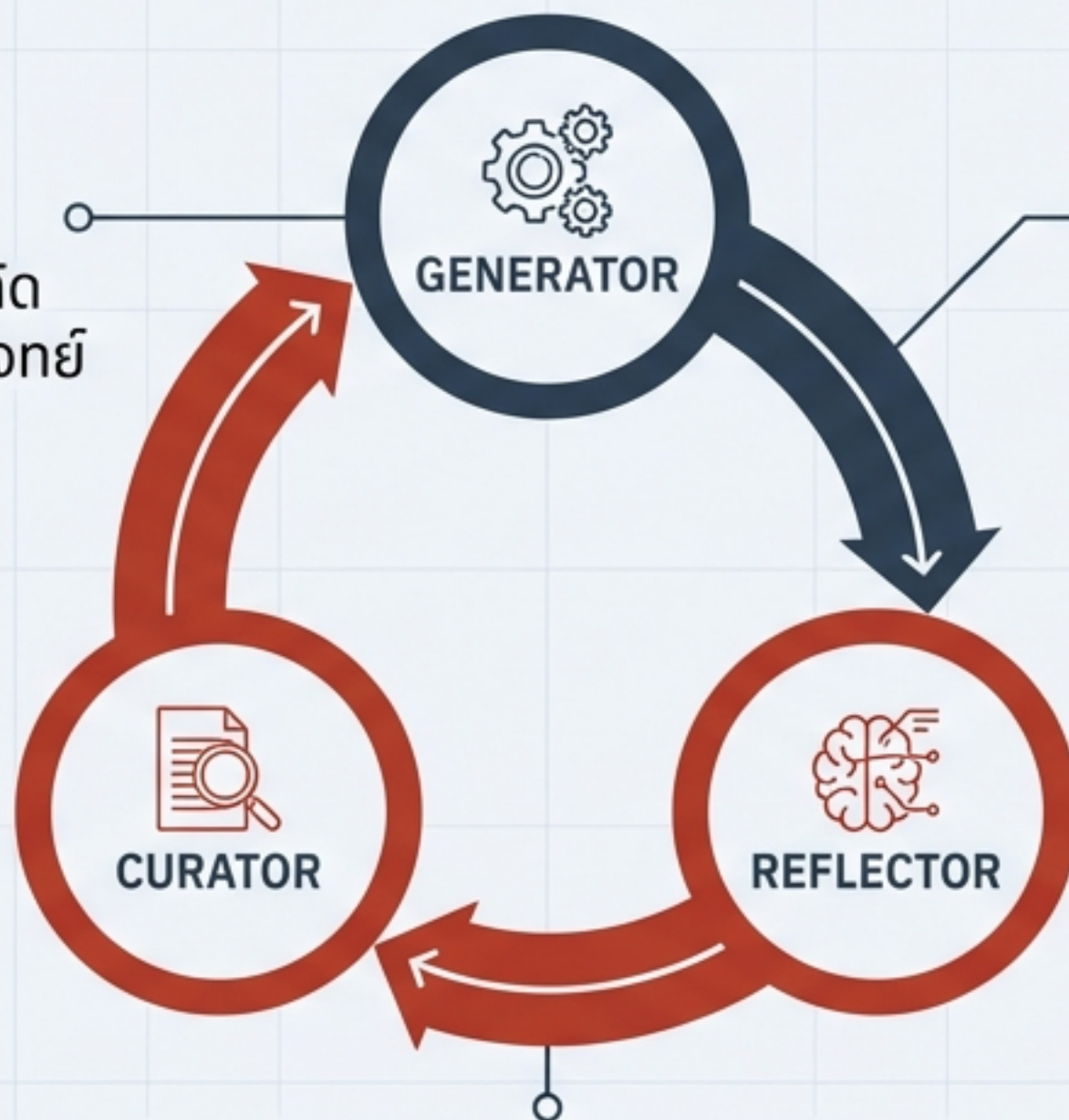


Delta Context
Items

สถาปัตยกรรม 3 ประสาน: Generator, Reflector และ Curator

1. Generator (ผู้ปฏิบัติ):

สร้างแนวทางการแก้ปัญหาและเขียนโค้ด (Reasoning Trajectories) เพื่อตอบโจทย์



3. Curator (ผู้ดูแล):

นำบทเรียนใหม่มาผสานเข้ากับ Playbook เดิม จัดการความซ้ำซ้อน และดูแลให้บริบทเป็นระเบียบ

2. Reflector (ผู้วิเคราะห์):

ตรวจสอบผลลัพธ์ วิเคราะห์ความผิดพลาด และกลั่นกรองเป็นบทเรียน (Insights)

The Reflector: เปลี่ยนความผิดพลาดเป็นบทเรียนอันทรงคุณค่า

- กลไกหลัก: Reflector ไม่ได้ทำหน้าที่แค่ตรวจถูก/ผิด แต่ทำการวิเคราะห์เชิงลึก (Deep Reflection)

ตัวอย่างสถานการณ์ (Example Scenario)

Input: โค้ดที่เขียนผิดพลาดโดยใช้การจับคู่ข้อความ (String Matching) แทน API วันที่

Bad Reflection: “โค้ดรับไม่ผ่าน” (ไม่มีประโยชน์)

ACE Reflector Insight: “อย่าใช้ String matching สำหรับการทำธุรกรรม; ให้ใช้ DateTime API แทนเพื่อความแม่นยำ”



INSIGHTS

The Curator: ผู้ดูแล 'Playbook' ให้คมชัดและทันสมัย

The Rewrite Trap



การเขียนบริบทใหม่ทั้งหมด
(Monolithic Rewriting)
ทำให้ข้อมูลหายและใช้เวลานาน

The ACE Solution



1. **Incremental Delta Updates:** อัปเดตเฉพาะส่วนต่าง (Delta) ไม่ต้องแก้ทั้งเล่ม
2. **Grow-and-Refine:** สะสมความรู้ใหม่เข้าไปเรื่อยๆ แล้วค่อยทำการ 'จัดระเบียบ' (Pruning) เมื่อจำเป็น

Result: บริบทมีความต่อเนื่อง ไม่สูญหาย และปรับตัวได้ตามสถานการณ์

เจาะลึกโครงสร้าง 'Playbook' (คู่มือกลยุทธ์)

Strategies & Hard Rules:

กฎเหล็กที่ห้ามละเมิด (เช่น การจัดการ Time-sensitive transactions)

Useful Code Snippets:

ตัวอย่างโค้ดที่พิสูจน์แล้วว่าใช้งานได้จริง (เช่น การใช้ defaultdict)

Troubleshooting: วิธีแก้ปัญหาที่พบบ่อย (Common Pitfalls)

The Generated ACE Playbook on AppWorld

STRATEGIES AND HARD RULES

[shz-00009]

When processing time-sensitive transactions involving specific relationships: always resolve identities from the correct source app (phone contacts), use proper datetime range comparisons instead of string matching, and verify all filtering criteria (relationship + time) are met before processing items. This ensures accurate identification and processing of the right transactions.

USEFUL CODE SNIPPETS AND TEMPLATES

[code-00013]

For efficient artist aggregation when processing songs, use defaultdict(list) to map song titles to artist names:

```
from collections import defaultdict
artist_map = defaultdict(list)
for song in songs:
    artist_map[song['title']].extend([artist['name'] for artist in song['artists']])
```

TROUBLESHOOTING AND PITFALLS

[ts-00003]

If authentication fails, troubleshoot systematically: try phone number instead of email as username, clean credentials from supervisor, check API documentation for correct parameters etc. Do not proceed with workarounds.

โครงสร้างนี้ถูกออกแบบมาให้อ่านง่ายทั้งสำหรับมนุษย์และ AI

ชัยชนะของรุ่นเล็ก: ACE พา DeepSeek เอาชนะ GPT-4

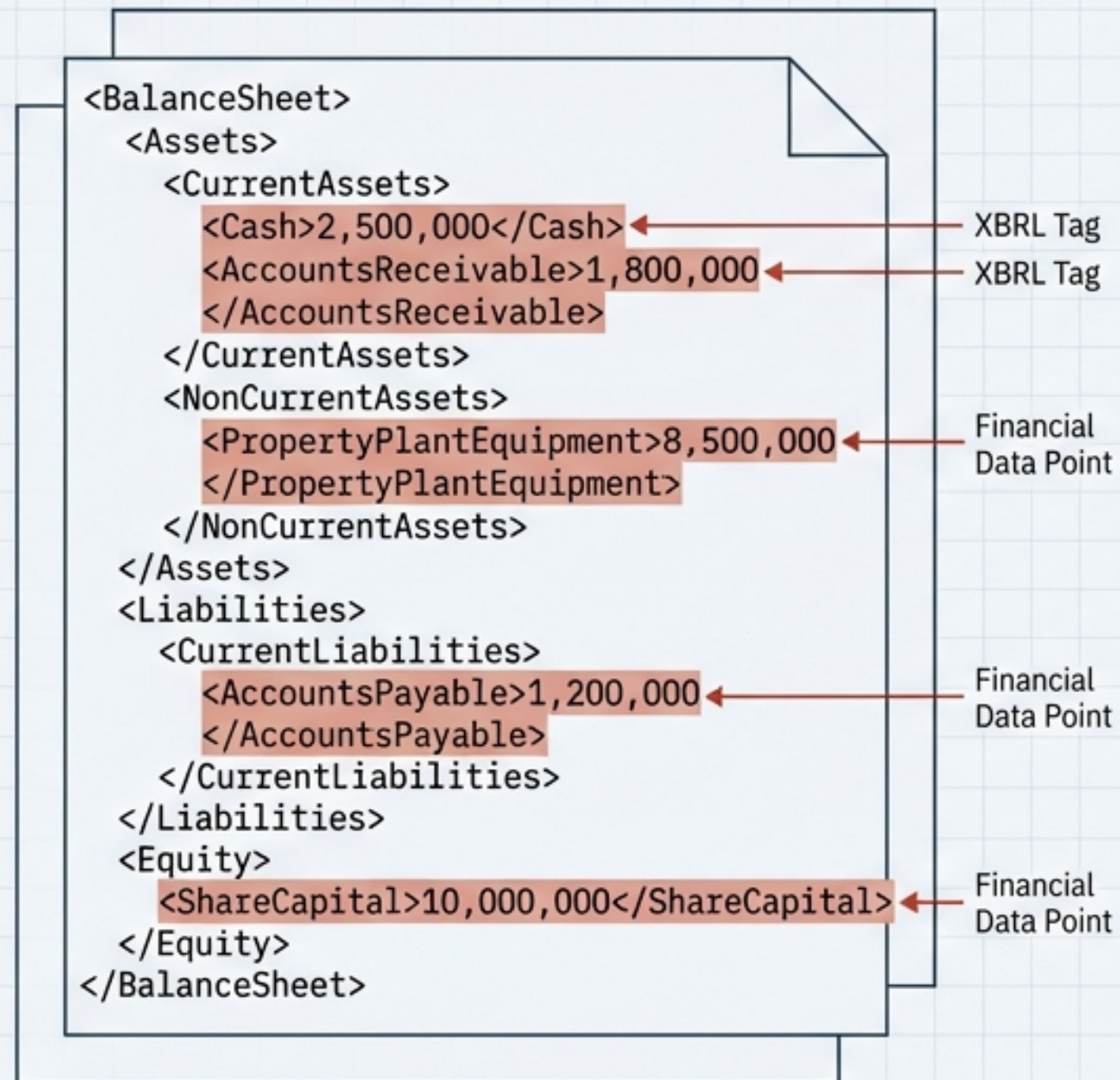
Method	LLM	Date	Test-Normal (TGC)	Test-Challenge (TGC)
IBM CUGA	GPT-4.1	2025-07-12	73.2	57.6
ReAct (with ACE/DeepSeek)	DeepSeek-V3.1	2025-07-26	48.8	58.2

**DeepSeek-V3.1
(Open Source) +
ACE $\approx$ IBM-CUGA
(GPT-4.1)**

**ชนะในระดับ
Challenge
(Hard Tasks)**

+10.6%
**ประสิทธิภาพ
เพิ่มขึ้นเทียบกับ
Baseline**

ความเชี่ยวชาญเฉพาะด้าน: กรณีสึกษาทางการเงิน (FiNER)



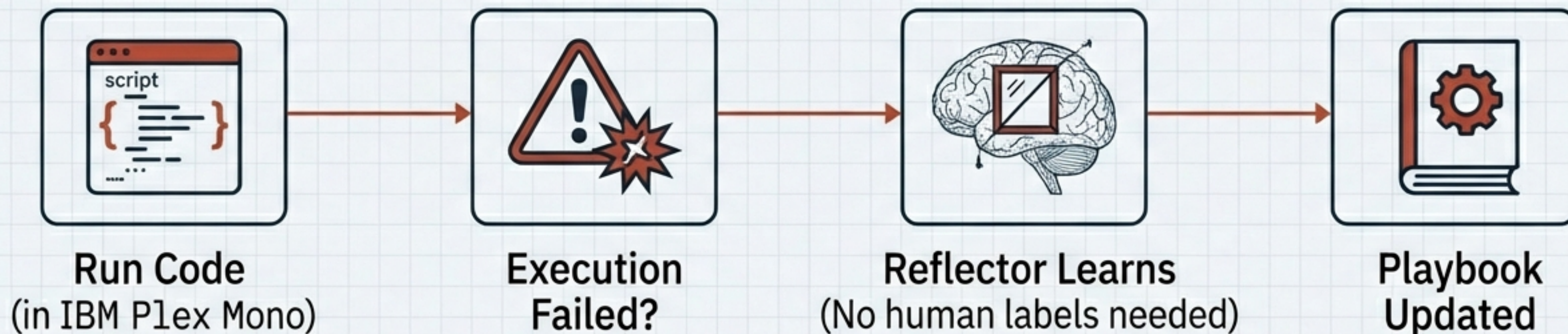
+8.6% Improvement in Finance Benchmarks

- งานด้านการเงินต้องการความแม่นยำสูงในระดับแท็ก (XBRL) ซึ่งการ 'ย่อความ' แบบเดิมมักทำให้ข้อมูลส่วนนี้หายไป
- ACE Playbook เก็บกฎเกณฑ์ทางบัญชีที่ซับซ้อนไว้ครบถ้วน ไม่มองว่าเป็นรายละเอียดส่วนเกิน

* Data Source: Table 2 from the research paper.

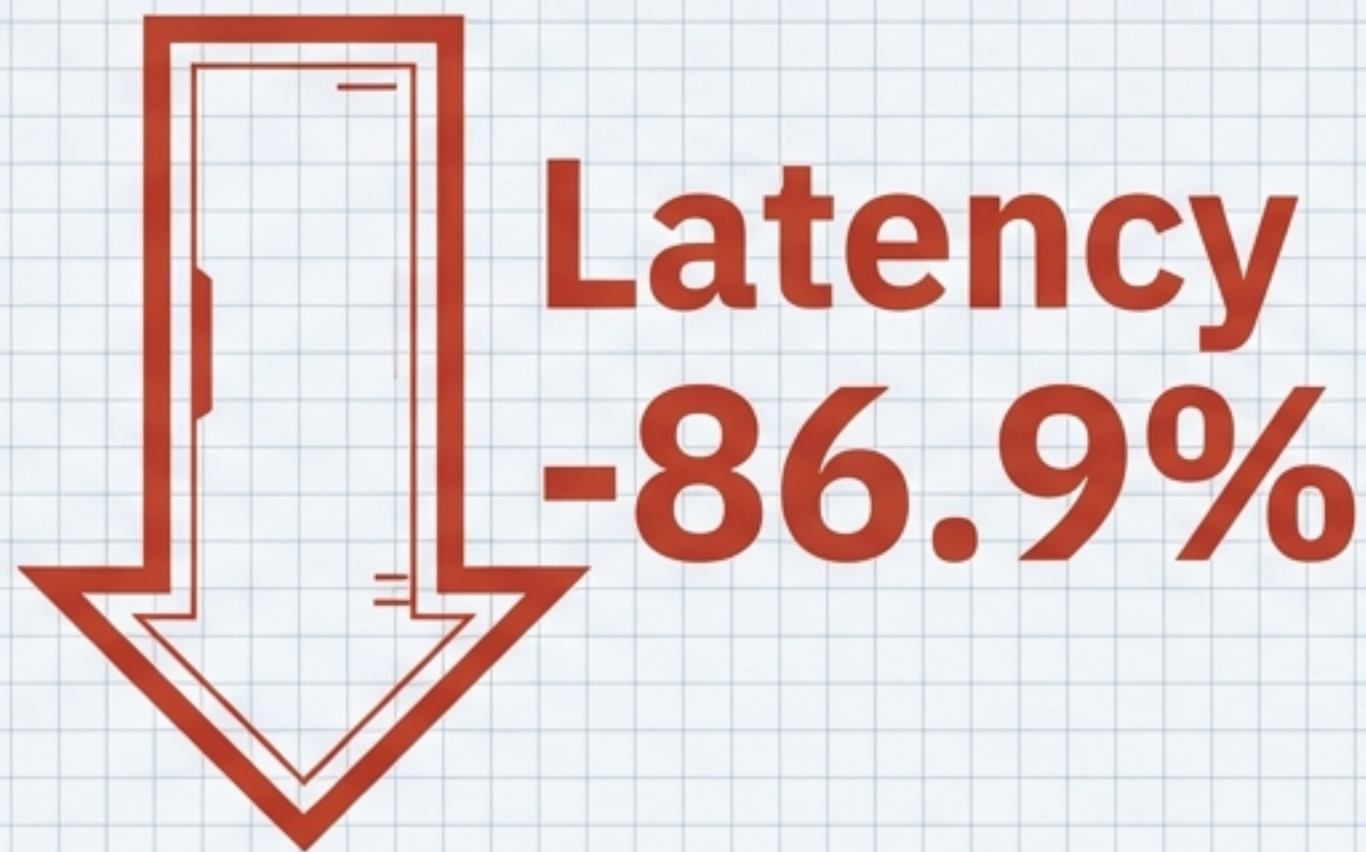
การเรียนรู้ไร้พี่เลี้ยง (Learning Without Teachers)

Natural Execution Feedback



สามารถเพิ่มประสิทธิภาพได้ **+14.8%** โดยไม่ต้องใช้ข้อมูลเฉลย (Ground Truth Labels) แม้แต่ตัวเดียว

ประสิทธิภาพที่เหนือกว่า: เร็วขึ้นและประหยัดขึ้น



ลดเวลาในการปรับแต่งลงมหาศาลเนื่องจากใช้
Delta Updates แทนการเขียนใหม่



ลดจำนวนครั้งที่ต้องรันโมเดลเพื่อทดสอบ
และประหยัดค่าใช้จ่าย Token

* ข้อมูลอ้างอิงจาก Table 4: Cost and Speed Analysis

อนาคตของ AI: การเรียนรู้ที่ไม่มีวันสิ้นสุด



Continuous Learning: ระบบที่ยังใช้งาน ยิ่งฉลาดขึ้น ตามขนาดของ Playbook ที่เติบโต



Machine Unlearning & Privacy: Playbook อ่านออกได้ (Readable) จึงสามารถ 'ลบ' ความรู้ที่ผิดพลาดหรือข้อมูลส่วนตัวออกได้ง่าย (Surgical removal)



Long-Context Serving: รองรับโมเดลยุคใหม่ที่มี Context Window ขนาดใหญ่ พร้อมเทคโนโลยี KV Cache

บทสรุป: ก้าวต่อไปของ AI Agents

- ✓ **Playbook > Summary:** บทบาทที่ดีต้องสะสมกลยุทธ์ ไม่ใช้การย่อความ
- ✓ **Architecture:** การแยกหน้าที่ (Generator-Reflector-Curator) ทำให้เกิดการเรียนรู้ที่มีประสิทธิภาพ
- ✓ **Self-Improvement:** ACE เปลี่ยน LLM ธรรมดาให้เป็น Agent ที่เรียนรู้จากข้อผิดพลาดได้เอง
- ✓ **Efficiency:** ประสิทธิภาพสูงขึ้น ในราคาและเวลาที่ลดลง

Comprehensive, evolving contexts enable scalable, self-improving AI systems.